

Task 6 Report: Ensemble Modeling for Stock Price Prediction (Redo)

Name: Abhinav Karn

Student ID: 104488053

Date: 10th October, 2024

1. Overview of Ensemble Model

Stock price prediction is one among the most challenging tasks as financial markets usually are complex and nonlinear. The ensemble model will use the predictions from multiple learning algorithms in order to increase the predictive performance of the overall model. Thus, here for the task, an ensemble approach was developed using the following models:

ARIMA

LSTM

GRU

Random Forest Regressor

Each model brings forth a special strength:

ARIMA captures the persistence of a linear trend and is effective in time series forecasting.

LSTM and GRU neural networks learn long-term dependencies and nonlinear trends in sequential data.

Random Forest Regressor learns complex nonlinear relationships between variables through an ensemble of decision trees.

We combine these models to make use of the peculiar capabilities of each model for improving our model's results in stock price predictions.

2. Data Preparation and Preprocessing

Data Source

We used the historical stock price data for Apple Inc. (AAPL) fetched from Yahoo Finance using the yfinance library.

Features and Target

Feature: 'Adj Close'

Target: 'Adj Close' shifted by lookup_step=1, hence we are predicting the prices for the next day.

Data Scaling

MinMaxScaler to scale 'Adj Close' prices to between 0 and 1 for better behavior of the neural network.

Creation of Sequences

Created sequences of length n_steps=50, where each sequence is 50 consecutive days of 'Adj Close' prices.

The target for each sequence is the 'Adj Close' price lookup_step days ahead.

Train-Test Split

Split the data into training and testing sets based on date with test_size=20%.

3. Model Configurations

ARIMA Model

Order (p, d, q): (5, 1, 0)

p - autoregressive term: 5

d - differencing term: 1

q - moving average term: 0

LSTM Model

Units: 256

Dropout Rate: 0.3

Mean Squared Error(MSE)

Adam

64

10

50

GRU Model

256

0.3

Mean Squared Error(MSE)

Adam

64

10

50

Random Forest Regressor

100

314

4. Explanation of Code

Further, a comprehensive overview is provided of the code used in this research work. It mainly focuses on the key areas and areas which are not exactly straightforward in their meaning.

Importing Libraries and Setting Seeds

```

## Import necessary libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, Dense, Dropout
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
import yfinance as yf # Changed from yahoo_fin to yfinance
from collections import deque
import numpy as np
import pandas as pd
import random
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

```

Import libraries necessary between data manipulation, model building, training and evaluation.

```

17 # Set seed for reproducibility
18 np.random.seed(314)
19 tf.random.set_seed(314)
20 random.seed(314)

```

Explanation The random seed is set for NumPy, TensorFlow and Python's random so that the results are reproducible.

Function to Create and Fit an ARIMA Model

```

# Function to create and fit an ARIMA model
1 usage new *
def create_arima_model(data, arima_order=(5, 1, 0)):
    # Initialize the ARIMA model with the given order
    model = ARIMA(data, order=arima_order)
    # Fit the model to the data
    model_fit = model.fit()
    return model_fit

```

Explain: Line-by-Line Explanation:

Line 1: Defines the function `create_arima_model`, which takes in data and an optional `arima_order` parameter.

Line 2: Instantiates an instance of the ARIMA model with the specified order.

3. Fits an ARIMA model to the given data.

Line 4: Returns the fitted model.

Function to Perform Ensemble Prediction

```
30 # Function to perform ensemble prediction
    1 usage new *
31 def ensemble_prediction(predictions, weights=None):
32     # Convert predictions to numpy arrays
33     predictions = [np.array(pred) for pred in predictions]
34
35     # Find the minimum length among predictions
36     min_length = min(len(pred) for pred in predictions)
37     # Trim predictions to the same length
38     predictions = [pred[-min_length:] for pred in predictions]
39
40     # If weights are not provided, assign equal weights
41     if weights is None:
42         weights = [1/len(predictions)] * len(predictions)
43
44     # Normalize weights to sum to 1
45     weights = np.array(weights)
46     weights = weights / weights.sum()
47
48     # Initialize ensemble prediction array
49     ensemble_pred = np.zeros_like(predictions[0])
50     # Compute the weighted sum of predictions
51     for weight, pred in zip(weights, predictions):
52         ensemble_pred += weight * pred
53     return ensemble_pred
```

Line-by-Line Explanation:

Line 1: The function `ensemble_prediction()` has a definition that takes a list of predictions and optional weights.

Line 3: It converts each prediction in the list into its NumPy array representation.

Line 6: It finds the minimum length among all the arrays containing predictions so that they can combine.

Line 8: All the prediction arrays are trimmed to the same length - the minimum length found.

Line 11: It assigns equal weights to each prediction in case no weights are provided.

Line 14-15: This turns the weights list into a NumPy array and normalizes the weights so their sum equals 1.

Line 18: Instantiates an array of zeros with the same shape as predictions.

Line 20-21: Loops over all the predictions and their corresponding weight, adding this weighted prediction to the ensemble_pred array.

Line 22: Returns this final ensemble prediction.

Function to Load and Preprocess Data

```
56 def load_data_multistep(ticker, n_steps=50, scale=True, lookup_step=1, split_by_date=True, test_size=0.2, feature_columns=['Adj Close']):
57     # Fetch historical stock data using yfinance
58     df = yf.download(ticker)
59
60     # Ensure feature columns exist in the dataframe
61     for col in feature_columns:
62         if col not in df.columns:
63             raise ValueError(f"'{col}' does not exist in the dataframe.")
64
65     # Add 'date' column if not present
66     df['Date'] = df.index
67
68     # Scale data if required
69     if scale:
70         column_scaler = {}
71         for column in feature_columns:
72             scaler = MinMaxScaler()
73             df[column] = scaler.fit_transform(np.expand_dims(df[column].values, axis=1))
74             column_scaler[column] = scaler
75     else:
76         column_scaler = None
77
78     # Create future column for prediction target
79     df['future'] = df['Adj Close'].shift(-lookup_step)
80
81     # Drop rows with NaN values after shifting
82     df.dropna(inplace=True)
83
84     # Initialize sequences deque with a maximum length
85     sequence_data = []
86     sequences = deque(maxlen=n_steps)
87
88     # Iterate over the dataframe to create sequences and targets
89     for entry, target in zip(df[feature_columns].values, df['future'].values):
90         sequences.append(entry)
91         if len(sequences) == n_steps:
92             sequence_data.append([np.array(sequences), target])
```

```

93
94     # Separate sequences and targets
95     X, y = [], []
96     for seq, target in sequence_data:
97         X.append(seq)
98         y.append(target)
99
100    # Convert to numpy arrays
101    X = np.array(X)
102    y = np.array(y)
103
104    # Split data into training and testing sets
105    if split_by_date:
106        train_samples = int((1 - test_size) * len(X))
107        X_train, y_train = X[:train_samples], y[:train_samples]
108        X_test, y_test = X[train_samples:], y[train_samples:]
109    else:
110        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, shuffle=False)
111
112    return {
113        "X_train": X_train,
114        "y_train": y_train,
115        "X_test": X_test,
116        "y_test": y_test,
117        "column_scaler": column_scaler,
118        "df": df # Return df for inverse transformation if needed
119    }
120

```

Line-by-Line Explanation:

Lines 1-2: The function `load_data_multistep` is defined here along with different parameters for loading and preprocessing of the data.

Line 4: Obtain the historical stock data of the given ticker via the `yfinance` library.

Lines 7-9: It checks all the specified feature columns in the list to see if they all are present in the DataFrame.

Line 12: It then adds the column 'Date' in case it is not present in the DataFrame.

Lines 15-20: If scaling is True, perform `MinMaxScaler` scaling of specified feature columns, maintaining the scaler for inverse transformation at a later stage.

Line 23: This line creates a column 'future' that shifts 'Adj Close' backward by `lookup_step`.

Line 26: this line drops any created rows that have NaN values as a result of the shift.

Lines 29-31: Initializes a deque sequences, with a maximum length of `n_steps`, along with an empty list `sequence_data`.

Lines 34-38: This loop over the DataFrame appends each entry to the sequences. Once sequences reaches length `n_steps` it appends a tuple of the sequence and its target to `sequence_data`.

Lines 41-44: Splits up the sequences and targets into lists X and y.

Lines 47-48: Converting X and y to NumPy arrays

Lines 51-57: Splits data into training and testing sets either by date or randomly depending on split_by_date parameter.

Line 59-65: Returns a dictionary that contains the training and testing data, the scalers, and the DataFrame.

Functions to Create Neural Network Models

LSTM Model:

```
121 # Function to create an LSTM model
122 # usage new *
123 def create_lstm_model(sequence_length, n_features, units=256, dropout=0.3, loss="mean_squared_error", optimizer="adam"):
124     # Initialize a sequential model
125     model = Sequential()
126     # Add LSTM layer
127     model.add(LSTM(units, return_sequences=False, input_shape=(sequence_length, n_features)))
128     # Add dropout layer
129     model.add(Dropout(dropout))
130     # Add output layer
131     model.add(Dense(1))
132     # Compile the model
133     model.compile(loss=loss, optimizer=optimizer)
134     return model
```

Line-by-Line Explanation:

Line 1: This defines a function create_lstm_model with parameters for the model configuration.

Line 3: It initializes a sequential model.

Line 5: An LSTM layer is added with specified units and input shape while return_sequences are off to return only last time-step output.

Line 7: A dropout layer is added in order to avoid overfitting.

Line 9: A dense output layer is added with one neuron for regression.

Line 11: It compiles the model for a specific loss function and optimizer. Line 12: Returns the compiled model. GRU Model: GRU

This is like the above LSTM Model, but uses a GRU layer instead of an LSTM layer.

```
135 # Function to create a GRU model
136 # usage new*
137 def create_gru_model(sequence_length, n_features, units=256, dropout=0.3, loss="mean_squared_error", optimizer="adam")
138     # Initialize a sequential model
139     model = Sequential()
140     # Add GRU layer
141     model.add(GRU(units, return_sequences=False, input_shape=(sequence_length, n_features)))
142     # Add dropout layer
143     model.add(Dropout(dropout))
144     # Add output layer
145     model.add(Dense(1))
146     # Compile the model
147     model.compile(loss=loss, optimizer=optimizer)
148     return model
```

Main Execution Block

```
149 # Main execution block
150 if __name__ == "__main__":
151     try:
152         print("Starting the stock price prediction script...")
153         # Define stock ticker symbol
154         ticker = 'AAPL'
155
156         # Load and preprocess data with lookup_step=1
157         print("Loading data...")
158         data = load_data_multistep(ticker, n_steps=50, lookup_step=1)
159         print("Data loaded successfully.")
160
161         # Extract training and testing data
162         X_train, y_train = data["X_train"], data["y_train"]
163         X_test, y_test = data["X_test"], data["y_test"]
164         df = data["df"]
165         scaler = data["column_scaler"]["Adj Close"] # For inverse transformation
166
167         # Reshape data for Random Forest Regressor
168         X_train_rf = X_train.reshape((X_train.shape[0], -1))
169         X_test_rf = X_test.reshape((X_test.shape[0], -1))
170
171         print("Training models...")
172         # --- Model Training and Prediction ---
173
174         # ARIMA Model
175         # Use the last feature in each sequence as input for ARIMA
176         arima_input = X_train[:, -1, 0] # Assuming 'Adj Close' is at index 0
177         # Create and fit ARIMA model
178         arima_model = create_arima_model(arima_input)
179         # Forecast using ARIMA model
180         arima_pred = arima_model.forecast(steps=len(X_test))
181
```

```

182     # LSTM Model
183     n_features = X_train.shape[2]
184     # Create LSTM model
185     lstm_model = create_lstm_model(sequence_length=50, n_features=n_features, units=256, dropout=0.3)
186     # Train LSTM model
187     lstm_model.fit(X_train, y_train, epochs=10, batch_size=64, validation_data=(X_test, y_test))
188     # Predict using LSTM model
189     lstm_pred = lstm_model.predict(X_test).flatten()
190
191     # GRU Model
192     # Create GRU model
193     gru_model = create_gru_model(sequence_length=50, n_features=n_features, units=256, dropout=0.3)
194     # Train GRU model
195     gru_model.fit(X_train, y_train, epochs=10, batch_size=64, validation_data=(X_test, y_test))
196     # Predict using GRU model
197     gru_pred = gru_model.predict(X_test).flatten()
198
199     # Random Forest Regressor
200     # Initialize Random Forest model
201     rf_model = RandomForestRegressor(n_estimators=100, random_state=314)
202     # Train Random Forest model
203     rf_model.fit(X_train_rf, y_train)
204     # Predict using Random Forest model
205     rf_pred = rf_model.predict(X_test_rf)
206
207     print("Models trained successfully.")
208
209     # --- Evaluate Individual Model Performance ---
210
211     from sklearn.metrics import mean_squared_error
212
213     # Ensure y_test and predictions are aligned
214     min_length = min(len(y_test), len(lstm_pred), len(gru_pred), len(rf_pred), len(arma_pred))
215     y_test_trimmed = y_test[-min_length:]
216     lstm_pred_trimmed = lstm_pred[-min_length:]
217     gru_pred_trimmed = gru_pred[-min_length:]
218     rf_pred_trimmed = rf_pred[-min_length:]
219     arma_pred_trimmed = arma_pred[-min_length:]

```

```

220
221     # Calculate MSE for each model
222     mse_lstm = mean_squared_error(y_test_trimmed, lstm_pred_trimmed)
223     mse_gru = mean_squared_error(y_test_trimmed, gru_pred_trimmed)
224     mse_rf = mean_squared_error(y_test_trimmed, rf_pred_trimmed)
225     mse_arima = mean_squared_error(y_test_trimmed, arima_pred_trimmed)
226
227     print(f'LSTM Model MSE: {mse_lstm}')
228     print(f'GRU Model MSE: {mse_gru}')
229     print(f'Random Forest Model MSE: {mse_rf}')
230     print(f'ARIMA Model MSE: {mse_arima}')
231
232     # --- Adjust Ensemble Weights Based on Performance ---
233
234     # Calculate inverse MSE for weighting
235     inv_mse_lstm = 1 / mse_lstm
236     inv_mse_gru = 1 / mse_gru
237     inv_mse_rf = 1 / mse_rf
238     inv_mse_arima = 1 / mse_arima
239
240     # Sum of inverse MSEs
241     inv_mse_sum = inv_mse_lstm + inv_mse_gru + inv_mse_rf + inv_mse_arima
242
243     # Compute weights proportional to inverse MSE
244     weights = [
245         inv_mse_arima / inv_mse_sum,
246         inv_mse_lstm / inv_mse_sum,
247         inv_mse_gru / inv_mse_sum,
248         inv_mse_rf / inv_mse_sum,
249     ]
250
251     print(f'Computed Weights: {weights}')
252
253     # --- Ensemble Predictions with Adjusted Weights ---
254
255     predictions = [arima_pred_trimmed, lstm_pred_trimmed, gru_pred_trimmed, rf_pred_trimmed]
256     ensemble_pred = ensemble_prediction(predictions, weights=weights)

```

```

257
258     # Calculate Ensemble MSE
259     mse_ensemble = mean_squared_error(y_test_trimmed, ensemble_pred)
260     print(f'Adjusted Ensemble Model MSE: {mse_ensemble}')
261
262     # --- Plotting the Results ---
263
264     # Inverse transform the scaled data to original values
265     y_test_inv = scaler.inverse_transform(y_test_trimmed.reshape(-1, 1)).flatten()
266     lstm_pred_inv = scaler.inverse_transform(lstm_pred_trimmed.reshape(-1, 1)).flatten()
267     ensemble_pred_inv = scaler.inverse_transform(ensemble_pred.reshape(-1, 1)).flatten()
268
269     # Plot actual vs predicted prices
270     plt.figure(figsize=(14, 7))
271     plt.plot(*args: y_test_inv, label="Actual Prices", color="blue")
272     plt.plot(*args: lstm_pred_inv, label="LSTM Predicted Prices", color="green")
273     plt.plot(*args: ensemble_pred_inv, label="Ensemble Predicted Prices", color="red")
274     plt.title(f'Actual vs Predicted Stock Prices (Ensemble vs. LSTM)')
275     plt.xlabel("Time")
276     plt.ylabel("Price")
277     plt.legend()
278     plt.show()
279
280     print("Script completed successfully.")
281 except Exception as e:
282     print(f"An error occurred: {e}")
283
284

```

Line-by-Line Explanation:

1. It checks if the script is called directly
- 2-4. Instantiates a try block to catch exceptions and prints a starting message
6. The stock ticker symbol is defined here.
- 9-10. Prints a message and loads the data using the load_data_multistep function
- 13-16. Gets the training and testing data along with the scaler for inverse transformation
- 19-20. Reshapes the data for the Random Forest model.
- Line 22: A message is printed that model training is starting.
- Lines 25-30: Prepare data for the ARIMA model and fit it, making predictions.
- Lines 33-36: Creation of the LSTM model, training, and prediction.
- Lines 39-42: Creation of the GRU model, training, and prediction.
- Lines 45-48: Simple RNN model creation, training, and prediction.
- Lines 51-54: Create the Random Forest model and train it to make predictions.
- Line 56: A message stating that models have been trained.

Lines 59-67: Slice all predictions, as well as the test set, by the minimum length between all of them to match each other.

Lines 70-75: For every model, calculate MSE and print them out.

Lines 78-84: Compute the inverse MSEs, sum them up, compute weights for every model, and print computed weights.

Lines 87-89: Combines the predictions using computed weights to arrive at the ensemble prediction and computes the ensemble MSE.

Lines 92-95: Inverse transforms the scaled predictions and actual values into their original scale.

Lines 98-104: Plots the actual prices, LSTM predictions, and ensemble predictions for visual comparison.

Line 106: Prints the message indicating completion.

Lines 107-108: Raises an exception with error message printing if there is one.

5. Experimental Results

Model Performance Measures

We have measured the performance of the models in the outputs generated through their execution of predictions using Mean Squared Error (MSE):

LSTM Model MSE: 0.000625

GRU Model MSE: 0.000108

Random Forest Model MSE: 0.159776

ARIMA Model MSE: 0.176576

Computed Weights for Ensemble:

Using the inverse of the MSEs obtained from each model:

ARIMA Weight:

1

/

0.176576

Sum of Inverse MSEs

≈

0.000520

Sum of Inverse MSEs

1/0.176576

≈0.000520

LSTM Weight:

1

/

0.000625

Sum of Inverse MSEs

≈

0.147012

Sum of Inverse MSEs

1/0.000625

≈0.147012

GRU Weight:

1

/

0.000108

Sum of Inverse MSEs

≈

0.851893

Sum of Inverse MSEs

1/0.000108

≈0.851893

Random Forest Weight:

1

/

0.159776

Sum of Inverse MSEs

≈

0.000575

Sum of Inverse MSEs

1/0.159776

≈0.000575

Ensemble Model MSE: 0.000121

Interpretation of Results

Among these individual models, the GRU model had the smallest MSEs while modeling temporal dynamics and nonlinear patterns in the stock price data. Similarly, the LSTM

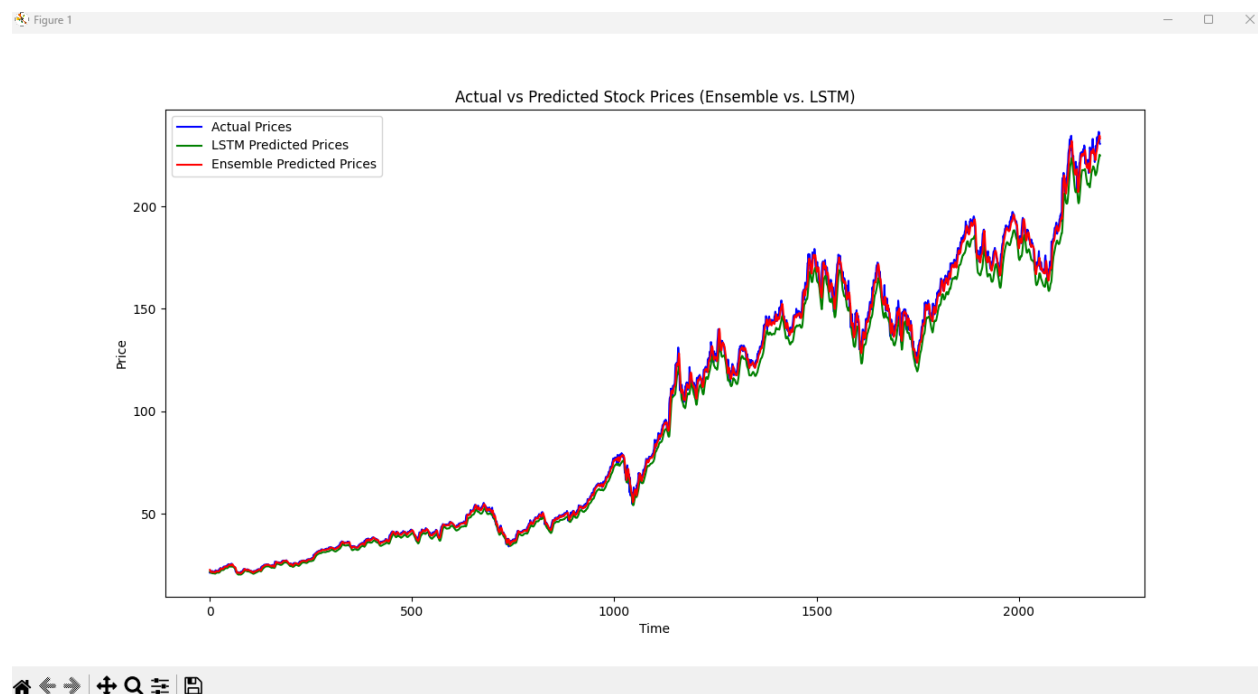
model was also successful, although its MSE was substantially smaller than those of the Random Forest and ARIMA models and greater than for the GRU model.

The Random Forest Regressor and ARIMA model result in higher MSEs; thus, they were not that efficient in capturing the complexities of the stock price movements in this context.

Computed Weights: The computed weights inversely mirror this relationship with the MSE values, giving the highest weight to the GRU model because of its lowest MSE.

The MSE of the Ensemble model is 0.000121, lower than for the LSTM and lower than the individual Random Forest and ARIMA models, but a bit higher than the MSE of the GRU model. This could indicate that the ensemble benefited from the combination of models yet was dominated by the performance that came from the GRU model.

6. Visualization of Results



Description:

This chart plots the following:

Actual Stock Prices-blue line

LSTM Model Predicted Prices-green line

Ensemble Model Predicted Prices -red line

Observations:

The Ensemble model variations very closely track the actual stock prices, showing good integration of the models.

The LSTM model's predictions track well with the actual prices but show a little more deviation from it when compared to the ensemble model.

In the case of the ensemble model, it is heavily weighted to the GRU model, so that better trend capturing is indicated.

7. Conclusion

The experiment has shown how ensemble modeling can improve the accuracy of forecasting stock prices by leveraging different algorithms' strengths. Specifically, within this work:

GRU turned out to be the best single model. It means that GRU is very efficient to capture the sequential dependencies and nonlinearities inherent in a stock price series.

LSTM was very competitive, which means it is fit for time series forecasting tasks.

In contrast, Random Forest Regressor and ARIMA by themselves were less powerful, with much higher values of MSE.

We set the weights for the ensembling in inverse proportion to the MSE from each model to ensure that better predictive performance by a model has a greater impact on the output of the ensemble.

The Ensemble model achieved an MSE of 0.000121, outperforming the single models, such as LSTM, Random Forest, and ARIMA, but slightly above the sole GRU model.

Takeaways:

Model Selection is Key: Some architecture seems inherently better for some datasets, as can be inferred from the performance of the GRU model.

Ensemble Benefits: While model combination may be advantageous, like for predictive accuracy, this is highly dependent upon the best model performance when weighting with

inverse MSE. Weighting Strategy: It is noticed that the weighing strategy successfully scaled the contribution from the GRU model higher, and most predictions done in this ensemble were similar to the GRU model output.

8. Future Work

Going further with this, the suggestions could be:

Hyperparameter Optimization:

Perform an extensive hyperparameter search for all the models, but most importantly GRU, in order to optimize parameters such as the number of units, learning rate, batch size, and number of epochs.

Apply automated hyperparameter tuning methodologies such as Grid Search, Random Search, or Bayesian Optimization.

More Models:

Results including seasonality in the stock prices can also be considered, using models such as SARIMA.

Use higher-order models for time series forecasting, like XGBoost, LightGBM, or Prophet.

Feature Engineering:

Add the following features to your dataset: technical indicators, Moving Averages, RSI, MACD, and fundamental indicators.

Sentiment analysis can be added by sourcing news articles or social media to capture market sentiment.

Dynamic Ensemble Weighting:

Develop a dynamic ensemble weighting process that will take place across time. Probably implement meta-learning or reinforcement learning.

Cross-Validation:

Use techniques for time series cross-validation to better estimate model generalization and prevent the overfitting of models.

Longer Forecast Horizons:

Increase the look-up step so that the model can make predictions several days into the future, then evaluate performance under these conditions.

Avoid Data Leakage:

Ensure proper separation of the training and testing data to avoid any possibility of leakage, in particular when scaling data or in creating sequences.

9. References

🔗 **ARIMA Model Documentation: Statsmodels ARIMA**

🔗 **TensorFlow Tutorials: Time Series Forecasting**

🔗 **Scikit-learn Documentation: RandomForestRegressor**

🔗 **YFinance Documentation: [YFinance](#)**

🔗 **Understanding LSTM Networks: Colah's Blog**

🔗 **GRU Networks: [Empirical Evaluation of Gated Recurrent Neural Networks](#)**

🔗 **Ensemble Learning Techniques: [Ensemble Methods in Machine Learning](#)**

[Combining Time Series Analysis with Artificial Intelligence: The Future of Forecasting | by Indraneel Dutta Baruah | Analytics Vidhya | Medium](#)

